

```
'id' Integer AUTO_INCREMENT NOT NULL PRIMARY KEY,
'name' varchar(30) NOT NULL,
'guardian_name' varchar(30) NOT NULL,
'rollno' Integer NOT NULL,
'admission_date' datetime NOT NULL
)
;
COMMIT;
```

Since the tables are already created by `syncdb`, we can work on the data storage using data model objects. For debugging and learning purposes, we can make use of the Django shell interpreter:

```
slinux@hackbox:~/LFY/helloweb$ ./manage.py shell
Python 2.6.4rc2 (r264rc2:75497, Oct 20 2009, 02:55:11)
[GCC 4.4.1] on linux2
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from helloweb.student.models import StudentRegister
>>> student_object = StudentRegister()
>>> student_object.name = "S1"
>>> student_object.guardian_name = "G1"
>>> student_object.rollno = 1
>>> student_object.admission_date = '2009-11-01'
>>> student_object.save()
Now the object instance is saved in the table.
You can retrieve the object instances for StudentRegister data
model as the following.
>>> StudentRegister.objects.all()
[<StudentRegister: StudentRegister object>]
It returns a list of all instances of StudentRegister class.
```

```
If you need to get the object instance for a particular data
entry such that rollno=4,
>>> s = StudentRegister.objects.get(rollno=4)
If you need to remove it from database table,
>>> s.delete()
```

We will now modify the templates and views to support database interaction.

Add some more entries to the table by using `student_object = StudentRegister()` and save it using the `student_object.save()` method.

Replace the `student_list = {'S1':1, 'S2':2, 'S3':3, 'S4':4}` line in the view function with the following code that grabs data from the database.

```
from helloweb.student.models import StudentRegister
slist = StudentRegister.objects.all()
student_list = {}
for s in slist:
    student_list[s.name] = s.rollno
```

This will make it populate the `student_list` dictionary with entries from the database/`StudentRegister` instances.

Run the development Web server and see the output. You can see that the table is populated with data from the database. It would be the same data you have fed through the Django shell interface.

Finally, here is a task for you: Modify the template file and the view function `index()` to view all the data from the `StudentRegister` instances—i.e., the guardian's name, admission date, etc. Also write a page `http://127.0.0.1:8000/helloweb/post_data` and a `post()` view function so that we are able to insert data into the database from a Web interface. Use the same statements we used in the Django shell interface to implement it.

This article has covered most of the bits and bytes at the basic level of Django Web application development. Once you get started coding in Django, you will definitely fall in love with it. The Django project website has excellent documentation available. You should always use `http://docs.djangoproject.com/en/1.1/` as the primary reference. 

By: Sarath Lakshman

The author is a Hacktivist of Free and Open Source Software from Kerala. He loves working on the GNU/Linux environment and contributes to the PiTiVi video editor project. He is also the developer of SLYNIX, a distro for newbies. He blogs at www.sarathlakshman.info

Be Available ★ Grow Your Business

- ✕ **Unix / Linux Hosting since 2006**
- ✕ **99.9% Monthly Uptime SLA**
- ✕ **Anytime Phone Support**
- ✕ **Value for Money Assured**
- ✕ **Stable, Secure and Reliable**

Now
Offering

- **VPS Servers**
- **Dedicated Servers**
- **Joomla Hosting Plans**



TUXG Hosting
Web Hosting Wing of The UX Group

40-1-92, 1st Floor, Khanna Nagar
Benz Circle, VIJAYAWADA - 10
Call +91-866-6662368, +91-9989777150

<http://tuxghosting.com>